COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 15, 2026	**DATE SUBMITTED:**	Oct 20, 2026
NAME:	Alvarez, Joshua Caleb P. Garcia, Vince Joseph R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 **[Parallel Algorithms and Multithreading]**

1. PROCEDURE OUTPUT

ILO A	

Screenshot	**Code:**
	`#include <iostream>`
	`#include <thread>`
	
	`void print(int n, const std::string &str){`
	`  std::cout << "Printing integer: "<< n <<std::endl;`
	`  std::cout<<" Printing string: "<< str << std::endl;`
	`}`
	
	`int main(){`
	
	`  std::thread t1(print,10,"T.I.P.");`
	`  t1.join();`
	
	`  return 0;`
	`}`
	
	Output:
	`Printing integer: 10`
	`Printing string: T.I.P.`

Analysis	Using the **<thread>** library it makes it so the program can run multithreading task. By executing the **print()** function within a worker thread **t1** , the main thread passes the value **10** and **T.I.P** to **t1**and implements **t1.join()** to wait until the worker finishes printing

Table 13-1. Simple One-Threaded Example

COMPUTER ENGINEERING

QUEZON CITY

ILOB

Screenshot of the code/Output

ILOB > Multithread.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

OUTPUT:

```
PS C:\Users\TIPQC\Desktop
0 : T.I.P.1 : Competent

2 : Computer
3 : Engineers
```

```
PS C:\Users\TIPQC\Desktop\
0 : T.I.P.2 : Computer

1 : Competent
3 : Engineers
```

```
PS C:\Users\TIPQC\Desktop\
1 : Competent0 : T.I.P.

3 : Engineers
2 : Computer
PS C:\Users\TIPQC\Desktop\
```

```
PS C:\Users\TIPQC\Desktop\AL
3 : Engineers1 : Competent
2 : Computer

0 : T.I.P.
```

```
3 : Engineers0 : T.I.P.

1 : Competent
2 : Computer
PS C:\Users\TIPQC\Desktop\A
```

Analysis

In this code, a vector of string elements ("T.I.P.", "Competent", "Computer", "Engineers") is processed using a vector of threads to execute the print function concurrently. A loop iterates through the string vector, spawning a new thread for each index and pushing it into the threads vector to print its index and string value. Because the threads run asynchronously without explicit order or synchronization lock when writing to std::cout, the console outputs show different print orders across multiple runs. Finally, a second loop calls th.join() on

	every thread, ensuring the main thread waits for all print tasks to complete before exiting the program.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages.

- Multithreading is a technique in programming that allows a single program to split its task into smaller threads that execute simultaneously while sharing the process memory. Instead of processing instructions sequentially, it lets it leverage a multi-core cpu to handle overlapping operations more efficiently.

  The biggest advantage is a noticeable boost in the performance and responsiveness in the program. Heavy computations in the computer's background or any other network request can run on different threads, preventing the user interface of the main to freeze. Since threads share memory in the same process, switching between it can require less cpu overhead and memory than running multiple full apps.

  The biggest disadvantage of multithreading is the risk of the data that races when multiple threads modify the shared memory simultaneously, causing it to have bugs that are hard to debug. Fixing it requires precise synchronization, something like mutex locks, but using it wrong can trigger deadlocks where threads can freeze while waiting for each other. Another thing is creating threads carries inherent runtime overhead, which means it can slow down even the simple tasks

2. Rationalize the use of multithreading by providing at least 3 use-cases.

- Web servers use multithreading to handle incoming requests from multiple users at the same time without blocking new connections.

  Video game engines use separate threads for rendering graphics, calculating physics, and running audio so the game stays smooth.

  Image and data processing apps split large files into chunks across threads to process heavy filters or calculations much faster.

3. Differentiate between parallelism and concurrency.

- Concurrency is about managing multiple tasks by overlapping their execution, which can happen on a single CPU core through fast context switching.

  Parallelism is about executing multiple tasks at the exact same physical instance using separate CPU

cores.

A quick way to picture it is the coffee machine analogy: concurrency is two queues sharing one espresso machine, while parallelism is two queues using two separate machines at the same time.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

CODE

```cpp
#include <iostream>
#include <thread>

int counter = 0;

void add(int value) {
    counter += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    std::cout << "Value before T1.join(): " << counter << std::endl;
    T1.join();

    std::cout << "Value before T2.join(): " << counter << std::endl;
    T2.join();

    std::cout << "Value before T3.join(): " << counter << std::endl;
    T3.join();

    std::cout << "Final counter value: " << counter << std::endl;

    return 0;
}
```

OUTPUT

```
Final counter value: 60
```

Analysis:

In this code, a global **counter** variable starts at **0**, and as soon as **T1**, **T2**, and **T3** are created, they run the **add** function in the background simultaneously. The first **std**::**cout** prints **0** because **T1** hasn't finished running yet, but calling **T1.join()** forces the main program to wait for **T1** to complete (+**10**), followed sequentially by **T2.join()** **(+20)** and **T3.join()** **(+30)** to reach the final value of **60**. Because all three threads modify the exact same counter

variable without a mutex, this setup creates a classic data race

Code

```cpp
#include <iostream>
#include <vector>
#include <thread>

bool found = false;
int foundIndex = -1;

void linearSearch(const std::vector<int>& arr, int target, int start, int end) {
    for (int i = start; i < end; i++) {
        if (found) {
            return;
        }
        if (arr[i] == target) {
            found = true;
            foundIndex = i;
            return;
        }
    }
}

int main() {
    std::vector<int> arr = {12, 45, 7, 23, 89, 56, 34, 90, 67, 11};
    int target = 34;
    int mid = arr.size() / 2;

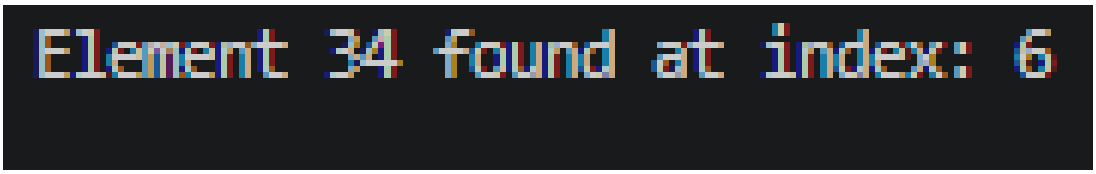
    std::thread T1(linearSearch, std::cref(arr), target, 0, mid);
    std::thread T2(linearSearch, std::cref(arr), target, mid, arr.size());

    T1.join();
    T2.join();

    if (found) {
        std::cout << "Element " << target << " found at index: " << foundIndex << std::endl;
    } else {
        std::cout << "Element " << target << " not found in array." << std::endl;
    }

    return 0;
}
```

OUTPUT



ANALYSIS:

In this code, we take the standard Linear Search algorithm and split the array into two halves to search them concurrently. Instead of doing a sequential loop through the whole vector, **res1** searches the first half while **res2** searches the second half at the exact same time using asynchronous execution. Whichever half contains the target number returns its index back to the main function, giving us the result without having the two search tasks interfere with each other.

CONCLUSION

Garcia	In this activity, I learned how multithreading lets programs run tasks simultaneously instead of waiting line by line, making better use of multi-core CPUs. I also got a solid grasp on how **.join()** keeps the main program from closing early, and why messing with shared memory without protection causes data races. The procedure built up the basics nicely, starting from spawning a single thread to running multiple worker threads in a vector loop, clearly showing how kicking off background execution saves time compared to standard sequential code. The supplementary activity then put this into practice, Part B showed asynchronous execution in action when counter stayed at **0** before **join(),** exposing how threads compete for global variables, while Part C proved its practical use by splitting Linear Search into two ranges so two threads could scan the array at the exact same time. Overall, I think I did pretty well getting the code to run and understanding how concurrent execution works under the hood, though my main areas for improvement are getting better at using mutex locks to stop data races and becoming more familiar with C++ compiler setup flags
Alvarez	In this activity, I gained a better understanding of how multithreading allows code to handle operations concurrently rather than executing everything sequentially, which really maximizes multi-core CPU usage. I learned how .join() ensures the main thread doesn't exit early before background tasks finish, as well as how accessing shared data without protection leads to data races. The initial procedure provided a clear progression, beginning with launching a basic thread and moving to handling multiple worker threads inside a vector loop, demonstrating how background threads speed up overall execution compared to traditional single-threaded programs. The supplementary tasks then demonstrated these concepts practically: Part B highlighted asynchronous behavior when counter remained 0 prior to calling join(), showing how unsynchronized threads collide over global variables, while Part C applied this by dividing a Linear Search across two sub-ranges so two threads could search the array simultaneously. Overall, I feel like I did a good job making the scripts compile properly and getting a clearer picture of how concurrent processing operates behind the scenes, though I definitely need to practice implementing mutexes to prevent data races and get more comfortable configuring C++ build settings.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar check and spell checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.